PA3 - AR Hit-Testing: write-up
Islamiya Okassova, ID 242322.

Variant: coffee table, multi mode (max 5), 8° tolerance, product faces the user.
Device: Samsung Galaxy S23 Ultra, Android 13, Chrome 155.0.8059.85, Google Play Services for AR 1.57.262640093.
Page: https://islamyiaok-cpu.github.io/PA3-AR-WebXR/

E1 - Reference spaces

Prediction: the camera starts near (0, 0, 0) in local. After walking 1 m forward it moves about 1 m horizontally. The same hit changes in viewer space and stays almost the same in local.

Result: at session start the log shows camera in local (start): 0.029, -0.066, -0.032. I marked two points on the floor 1 m apart, aimed the ring at the same floor point from both, and pressed Log pose each time.

	Log pose at point A	**Log pose after walking 1 m**
camera in local	0.337, 0.070, -0.080	-0.480, -0.019, -0.196
hit in viewer	0.000, 0.000, -2.460	0.000, 0.000, -1.638
hit in local	-1.977, -0.628, -0.538	-1.956, -0.645, -0.531

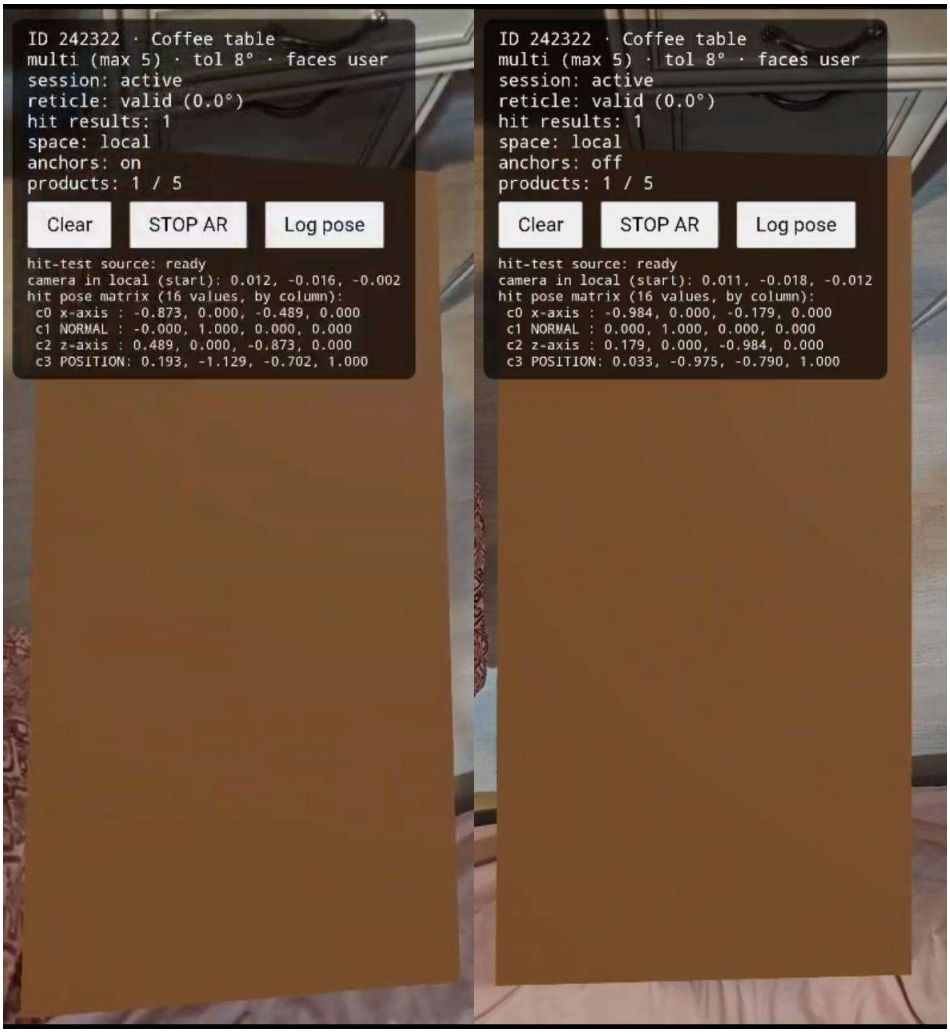


After the 1 m walk the camera position changed in local and the distance to the hit in viewer shrank from 2.460 m to 1.638 m, while the hit in local changed by only about 2.8 cm. The prediction is confirmed.

Explanation: the two poses describe the same physical point in two coordinate systems. viewer has its origin at the phone and looks along -z, so the hit-test ray always gives x = y = 0 and only the distance changes when I move. local has its origin fixed in the world at the session start, so the floor point keeps almost the same coordinates. The hit-test source is created from viewer because the ray must start at the phone and follow it. Results are read in local because the scene and the products live in local and must not move with the phone.

E2 - Checking Ch. 7

(a) Anchors on/off. Prediction: with anchors the offset after walking 3 m away and back is smaller than or equal to the offset without anchors. *Result:* the product was placed on a tape mark, then I walked 3 m away and back. Offset with anchors: on: 15 cm. Offset with anchors=off: 10 cm (one trial each).



The prediction was not borne out in this trial. Both offsets are small, and with one trial per mode a 5 cm difference lies within the run-to-run variation of tracking and of reading the tape, so it does not show that anchors are worse.

Explanation: the book states that getHitTestResults() uses the Spatial Anchors module to save each hit. That is not correct: a hit result is a pose valid for the current frame, and nothing is stored. An anchor is a separate object created with hit.createAnchor(). The system keeps updating its anchorSpace as tracking improves, and my code reads it every frame. Without an anchor the product keeps a fixed matrix in local and cannot follow later corrections of tracking.

(b) local-floor. Prediction: in local-floor the camera y is about the phone height above the floor; in local it is about 0. *Result:* in local, y was between -0.066 and 0.043 at session starts. With ?space=floor the log shows camera in local-floor (start): 0.004, 1.104, -0.008. *Explanation:* local puts the origin at the phone's start position; local-floor puts it on the estimated floor, so heights are measured from the floor. Hit testing works the same in both spaces, so local-floor does not improve it; it only changes the origin of the coordinates, which is convenient for floor-level content.



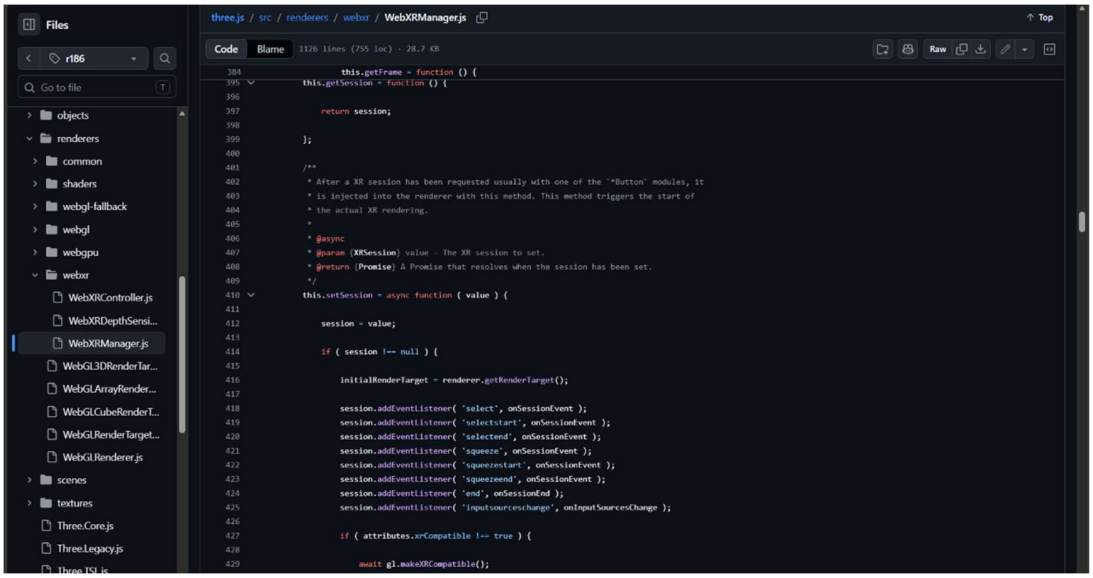
(c) setSession. The book's three critical tasks are: (1) connect the WebGL context to the XR session with gl.makeXRCompatible() and updateRenderState baseLayer: new

XRWebGLLayer(...), which the book wraps in its own setupWebGLLayer(); (2) renderer.xr.setReferenceSpaceType('local'); (3) renderer.xr.setSession(xrSession).

In WebXRManager.js (r186), setSession (line 410) is task 3 itself, and it already performs the other two:

- **Task 1.** Lines 427 - 429 call gl.makeXRCompatible() when the context is not already XR-compatible. Lines 451–453 create the XRWebGLLayer and call session.updateRenderState (baseLayer). If the browser supports projection layers, lines 496–498 do the same with createProjectionLayer and updateRenderState({ layers }).
- **Task 2.** The type set earlier with setReferenceSpaceType is used on line 526: referenceSpace = await session.requestReferenceSpace(referenceSpaceType).

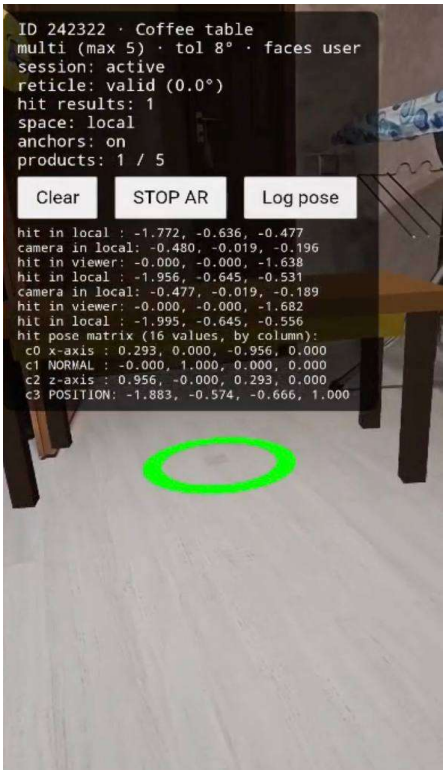
So setupWebGLLayer() is no longer needed.



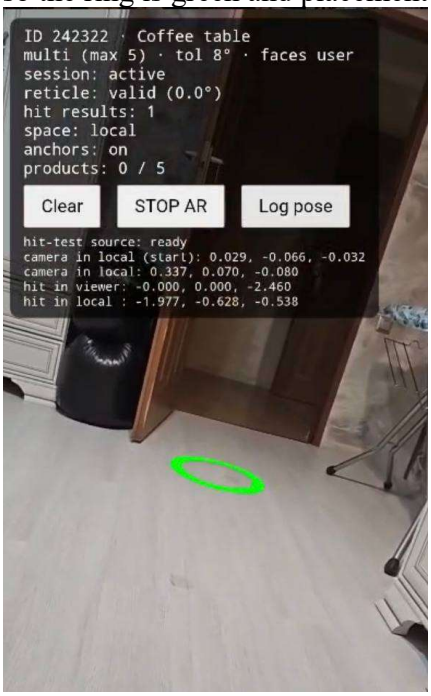
E3 - Reading the pose matrix

Hit pose matrix at one placement:

column	values	meaning
c0	0.293, 0.000, -0.956, 0.000	x-axis
c1	0.000, 1.000, 0.000, 0.000	surface normal
c2	0.956, 0.000, 0.293, 0.000	z-axis
c3	-1.883, -0.574, -0.666, 1.000	position



The normal is (m[4], m[5], m[6]) = (0, 1, 0), length 1, so tilt = acos(1/1) = 0°. That is within 8°, so the ring is green and placement is allowed (panel: valid (0.0°)).



Prediction: a flat floor reads about 0°, and a tilted surface reads close to its real angle. *Result:* on a vertical wall the panel shows too tilted (90.0°), the ring is red (by the formula: m[5] ≈ 0, acos(0) = 90°), and a tap placed nothing: products stayed 0.



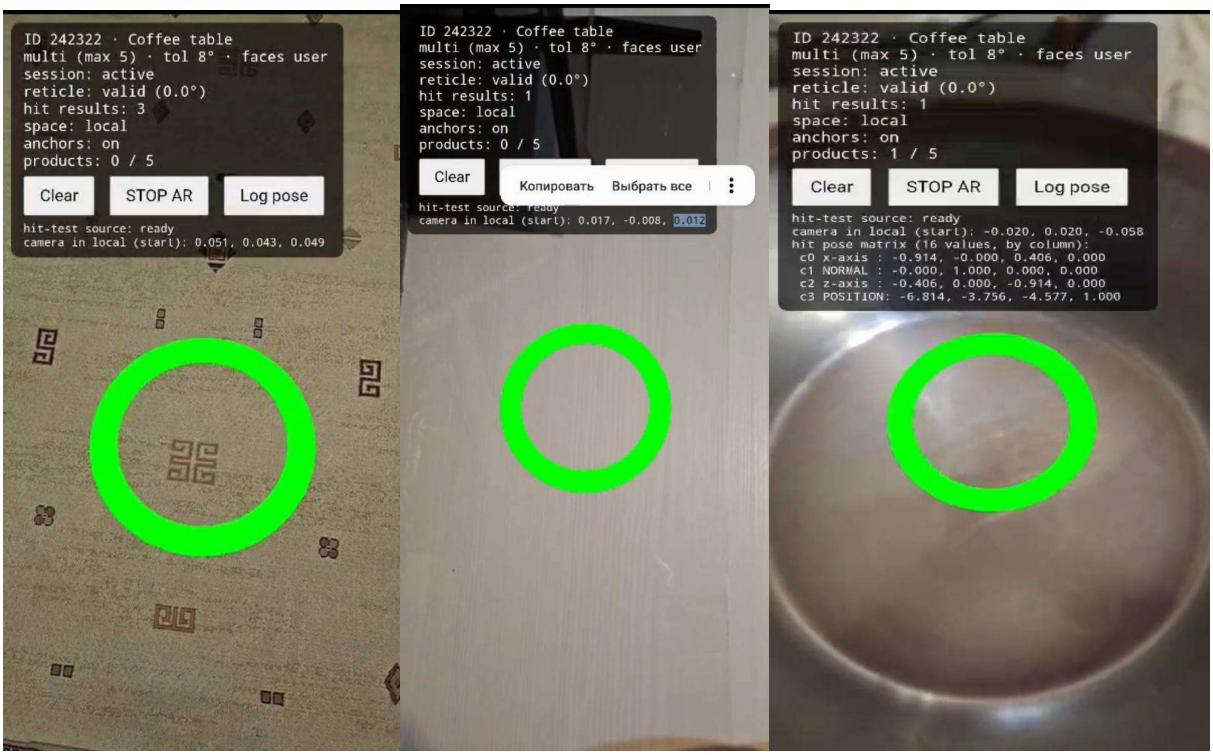
Explanation: the filter compares the normal with world up exactly as designed. Green within 8°, red outside, and red taps are ignored.

E4 - How the phone finds surfaces

The phone tracks itself with visual-inertial odometry. The camera detects feature points (corners and texture) and follows them from frame to frame. The gyroscope and accelerometer give fast motion estimates between frames, and the system fuses both. The parallax of the tracked points gives their 3D positions, and groups of points on one plane become a detected plane that the hit test can intersect.

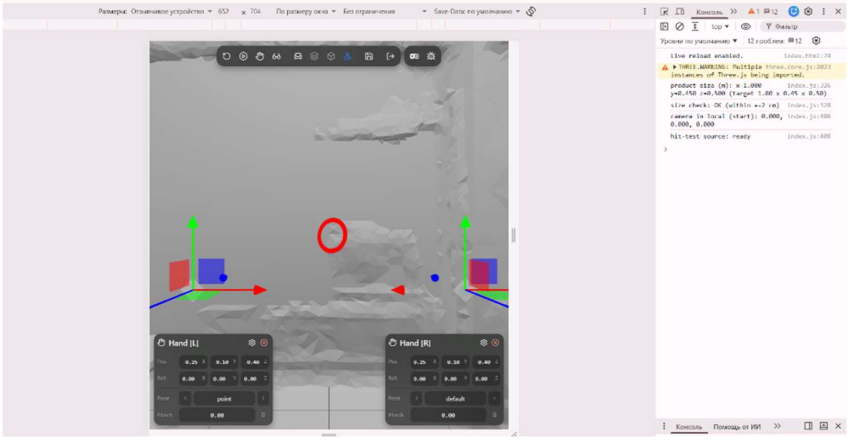
Prediction: patterned carpet fastest, a plain white table slower, a glossy surface hardest.

Surface	**Seconds to first green ring**	**Success**
Patterned carpet	1	yes
Metal object	4	yes
Plain white smooth table	18	yes



One trial per surface. The carpet was fastest and the plain white table slowest. The white table is uniform in colour and has almost no feature points to track. The carpet pattern gives many, so the plane was found almost at once. The metal object was in between: the edges and reflections of the objects around it most likely gave the camera enough features.

E5 - Scale check



The product size check in code gives x = 1.000, y = 0.450, z = 0.500 m (size check: OK). I placed the table next to a bottle measured with a tape at 24 cm and held the tape vertically beside the virtual table. The top of the table read 42 cm, so the height error is 42 − 45 = −3 cm (about 6.7 %).



E6 - Product preview for a retailer

Failure mode 1: on a plain white table the first hit took 18 s (carpet: 1 s). A shopper would give up. Fix: after 5 s without a hit, show “Move the phone slowly over a textured area or the floor”. Failure mode 2: on one phone I tried, START AR failed with NotSupportedError, and iPhones have no immersive-ar; the customer sees nothing. Fix: show the clear fallback message and offer a non-AR 3D viewer. A WebXR page can read, with permission, the device pose, hit-test results and anchors, and, only if requested, extras such as planes or raw camera images. My page requests only hit-test, anchors and dom-overlay, so it does not receive camera pixels. Users should be told that the camera is used for AR, that this page records and uploads nothing, and that they can stop at any time.

Development log

- Bug: on the S23, START AR showed the camera and a green ring (hit results: 1), but tapping the floor did not place a table (products: 0).
- Found: I re-read index.js and saw a second beforexrselect listener on the whole overlay root. The DOM overlay covers the screen, so it cancelled every tap, not only taps on the panel.
- Fix: removed that line and kept the listener on #panel only. Commit: e004c9a.